UCLouvain

epl

École polytechnique de Louvain

Direct device access from the smartNIC towards datacenters disaggregation

Optional subtitle

Author: **Nicolas JEANMENNE**
Supervisor: **Tom BARBETTE**
Readers: **Firstname LASTNAME, Firstname LASTNAME, Firstname LASTNAME**
Academic year 2025–2026
Master [120] in Computer Science

Abstract

TO-DO: complete abstract

Contents

Abstract **1**

1 Introduction **1**

2 Background **2**
2.1 Overview of concepts and definitions 2
2.2 Architectural challenges 3
2.2.1 Monolithic challenges 4
2.2.2 Disaggregation challenges 4
2.3 State-of-the-art designs and solutions 4
2.3.1 Zero-copy data path 4
2.3.2 Batched write 5
2.3.3 Split stacks 5
2.4 Unresolved challenges 6
2.5 Objectives 6

3 Measurements **7**
3.1 Disk performance baseline 7
3.1.1 Throughput 7
Sequential read 7
Sequential write 8
3.1.2 Latency 8

4 Appendix : performances script **i**

Bibliography **vi**

List of acronyms

AIO Asynchronous Input/Output. 7

ARM Advanced RISC Machine. 5

BDEV Block device. 7

CISC Complex Instruction Set Computer. 5

CPU Central Processing Unit. 4, 5

DA Disaggregated Architecture. 4

DPU Data Processing Unit. 5

FIO Flexible I/O tester. 7

Fish Friendly Interactive Shell. i

FPGA Field-Programmable Gate Array. 4

FS File System. 6

GPU Graphics Processing Unit. 4

HCA Host Channel Adapter. 5

HPC High-Performance Computing. 2

I/O Input/Output. 5–7

MA Monolithic Architecture. 4

MTU Maximum Transmission Unit. 5

NPF Network Performance Framework. i, 7

NVMe Non-Volatile Memory Express. 2, 3, 5, 6

NVMe-oF NVMe over Fabric. 2, 5

P2PDMA Peer-to-Peer Direct Memory Access. 5

PCI PCI Express. 5

QD Queue Depth. 7

RDMA Remote Direct Memory Access. 3, 4

RoCE RDMA over Convergent Ethernet. 3, 4

RTT Round-Trip Time. 5

SPDK Storage Performance Development Kit. 3, 6–8

SSD Solid State Drive. 2

TCP Transmission Control Protocol. 4–6

List of Listings

4.1 NPF script for sequential read throughput measurements with 4MB block size i
4.2 Automation script for running NPF experiments iii

Chapter 1

Introduction

TODO

Chapter 2

Background

This chapter provides an overview of disaggregated datacenter architectures and smartNIC-to-storage interactions. We will first give an overview of the main concepts and definitions used in the literature, then we will review the main architectural challenges and solutions proposed by researchers. Finally, we will list some unresolved challenges that we aim to solve in this thesis.

2.1 Overview of concepts and definitions

Hardware is evolving, notably through the transition from hard disk drives to Solid State Drive (SSD) devices and the decreasing cost of RAM. These trends have enabled researchers and companies to explore new approaches to High-Performance Computing (HPC). Modern SSDs use the Non-Volatile Memory Express (NVMe) protocol, a low-latency, high-throughput interface designed for flash storage. NVMe devices can be accessed over the network using NVMe over Fabric (NVMe-oF), which extends the Non-Volatile Memory Express command set to support network fabrics NT *what is a smartNIC? :d* (smartNICs, . . .) [1]–[3]. Such advancements made it possible to design new architecture designs, such as disaggregated datacenters.

This thesis uses *disaggregation* to denote an architecture in which hardware resources such as compute, memory, accelerators, and storage are split into pools (or nodes) interconnected through a high-speed network. This design allows resources to be allocated on demand, improving utilization and availability. Two levels are commonly distinguished. NT *Need more citations here* *Partial disaggregation* separates storage from compute and memory, which remain integrated together; this level has already been widely implemented since the early 2020s. *Full disaggregation* groups each resource type into dedicated clusters that communicate over the network. [4]. Figures 2.1 and 2.2 illustrate these two levels of disaggregation.

NT *Figures too small*

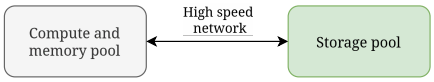


Figure 2.1: Partial disaggregation architecture

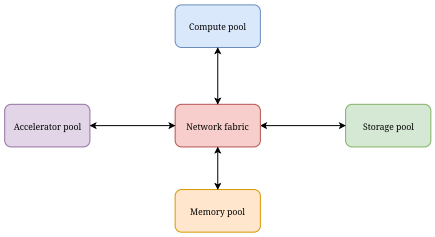


Figure 2.2: Full disaggregation architecture

High-speed network communication is a critical aspect of disaggregated architectures. Remote Direct Memory Access (RDMA) [5] [6] is a protocol that allows one computer to access memory on another computer without CPU involvement, providing high throughput and low latency. RDMA over Convergent Ethernet (RoCE) [7] is a variant of RDMA that operates over Ethernet networks, making it more widely compatible with existing infrastructure. NT *If we don't use it over Ethernet, what do we use it over then?*

Writing efficient storage applications involves understanding the role of Storage Performance Development Kit (SPDK) NT *is SPDK actually used in practice, need more citations here*. SPDK is a set of tools and libraries for building high-performance storage applications, particularly those that interact with NVMe devices. It provides a user-space driver that bypasses the kernel, allowing for lower latency and higher throughput compared to traditional kernel-based drivers [8]. This development kit will be used in this thesis to implement and evaluate our proposed solutions, as detailed in the next chapters. NT *Need more details about how SPDK works and why is it better that the Kernel, in which case is it better*

DRAFT NOTES

Sections below (until end of chapter) are draft notes and need to be rewritten and structured. TODOs :

- Rewrite and structure improvements
- Table of comparison of solutions

2.2 Architectural challenges

Architectural challenges can be grouped into two categories:

- Limitations of traditional monolithic servers, which motivate the transition toward disaggregated architectures.

- New challenges introduced by disaggregation, which break assumptions commonly made in earlier systems research and require new design approaches.

NT *I would focus more on your topic and what you are trying to solve*

2.2.1 Monolithic challenges

The main limitation of Monolithic Architecture (MA) is **resource stranding**. When one resource is exhausted (for example, memory), other resources on the same server may remain underutilized, leading to waste.

Another important issue in Monolithic Architecture (MA) is availability under failures. A single hardware failure can impact the whole server and make it unavailable.

Finally, hardware replacement and upgrades in Monolithic Architecture (MA) often require costly maintenance operations.

2.2.2 Disaggregation challenges

In disaggregated architecture (Disaggregated Architecture (DA)), the network becomes the central communication path between resource clusters. This creates strict latency and bandwidth requirements: CPU-storage communication typically tolerates millisecond-scale delays, whereas CPU-memory communication requires nanosecond-scale delays (around 100 ns) [4].

Current network stacks also expose strong flexibility-performance trade-offs. RDMA and RDMA over Convergent Ethernet (RoCE) offer high performance but limited flexibility due to hardware coupling. In addition, RDMA can suffer from head-of-line blocking, congestion, and vendor lock-in because its requirements are not always satisfied by commodity hardware. By contrast, Linux TCP is more flexible but delivers lower performance and less efficient CPU utilization [9].

2.3 State-of-the-art designs and solutions

2.3.1 Zero-copy data path

The term *zero-copy* refers to data transfer between devices without requiring data replication by the CPU, thereby avoiding redundant copies and reducing CPU usage. Skiadopoulos, Xie, Zhao, *et al.* [9] designed a new prototype called *ZeroNIC* which can zero-copy data between NIC and accelerators. Its main advantage is that is it agnostic of the accelerator type and can run on CPU, GPU, FPGA, . . . NIC hardware split header and payload, transferring the latter directly to the receiver applications buffers without intermediates copies.

Concerning NVMe storage, Sun, Zhang, Shan, *et al.* [10] focused on NVMe-oF target offloading which extends zero-copy mechanism, transferring data to the Data Processing Unit (DPU)'s Host Channel Adapter (HCA) via peer-to-peer PCI Express (PCI) communication (Peer-to-Peer Direct Memory Access (P2PDMA)) and thus creating a new kind of zero-copy data path.

2.3.2 Batched write

Batched write mechanism retains write operations until an arbitrary threshold is achieved, then all these operations are run consecutively while a group commit ensures atomicity of operations. The goal is to improve throughput and reduce the accelerator CPU load. While it is not technically a contribution brought by disaggregation, its evolution into such systems is still relevant to be mentioned.

2.3.3 Split stacks

NT *Should be more high-level there, you need to explain the ideas* NT *I would add a diagram here*

Split stacks are networking stacks divided in several planes, each one optimized for a specific task. IO-TCP from Kim, Ng, Gong, *et al.* [11] separate the traditional Linux TCP stack into a stack running on the host CPU, performing control operations, and thus called the control plane. Such operations are complex and benefits from features of the Complex Instruction Set Computer (CISC) CPU (for example branch prediction, vectorized instructions, . . .) NT *Pretty sure ARM also has those now :d*. The host stack extends the mTCP [12] API with methods to offload file opening, writing, and closing, such that applications have the most minimal changes to do in order to run itself on IO-TCP.

The data plane handles all operations related to data packet preparation and transfer. These operations are simpler in comparison of the control plane and then fit better the Advanced RISC Machine (ARM) cores of the smartNIC. They also benefit from being stateless which suits the usage of the smartNIC. File and disk Input/Output (I/O) are offloaded to otherwise it would interfere with the control plane on host CPU. Data plane operations are virtually performed and enable a mechanism of SEND / ECHO which translate in multiple Maximum Transmission Unit (MTU) packets transfer.

NT *This is not detailed enough, either go deeper or remov this part* Many difficulties due to the split introduce themselves, like Round-Trip Time (RTT) measurement, retransmission, and error handling, but are essentially resolved with the set of commands provided by the implementation.

2.4 Unresolved challenges

There are some challenges still left open upon reading papers in **section 2.3**. This section aims to list them, with the goal to define a clear approach for working on the master's thesis as well as a precise problem to solve.

IO-TCP [11] lacks of dynamic synchronization. This because authors assumes that files don't change during the content delivery service. In their solution, both host and smartNIC share the same filesystem, but any change on the host-side File System (FS) isn't propagated to the smartNIC stack.

Another problem is that file reading on NVMe disks is done with direct I/O on a regular ext4 file system. SPDK reaches the best performance, but support for file system isn't very developed for now. NT *Citation here*

Next, with a smaller amount of connection (typically < 4), Linux TCP stack outperforms IO-TCP by over 1.5 times.

Batched writes used in Scalio [10] to improve throughput of write operations come at the cost of latency increase. This can lead to non-negligible issues in environment where low latency is critical.

2.5 Objectives

This paper will focus on fully disaggregated datacenter, especially on interaction between acrshortnvme storage pools and smartNIC. One of our objectives is to allow SmartNICs to communicate directly with the storage without involving any compute pool.

Chapter 3

Measurements

This chapter will cover the measurements and evaluation steps from the baseline to the implemented solution. The performances evaluation focus mainly on two metrics : throughput and latency. Experiments could be make thanks to Network Performance Framework (NPF) [13]

3.1 Disk performance baseline

The first step in the measurements process is to evaluate the Input/Output (I/O) baseline of the disk. We'll use `Flexible I/O tester (FIO)` [14] with `libaio` [15] engine to test the throughput of the Linux kernel. Then we'll use `SPDK` own BDEV Asynchronous Input/Output (AIO) engine.

3.1.1 Throughput

Throughput has been evaluated for different file size, block size and I/O depth. I/O QD is set to 1, 16 or 32 and aggregated on mean. File size is set to 4MB, 10MB, 100MB and 1GB.

Sequential read

Figure 3.1 depict the throughput over 4kB and 4MB block size. Strictly speaking of maximum throughput, SPDK sequential read only outperforms Linux kernel by ≈ 1.282 with respectively 7.11 GB/s *versus* 7.02 GB/s. The most significant improvement are for 4MB and 10MB block size where SPDK doubles the throughput of Linux kernel. For 4MB block size, SPDK mean sequential read throughput is 2.1 GB/s while Linux kernel is 1.3 GB/s. For 10MB block size, SPDK mean sequential read throughput is 5.59 GB/s while Linux kernel is 2.76 GB/s. Figure 3.2 follows the same trend, although there is more variability in the results.

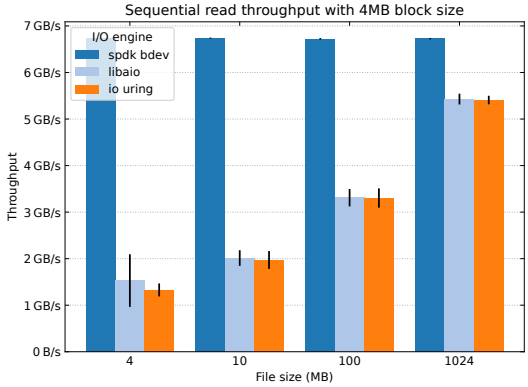


Figure 3.1: Throughput of sequential read operations with 4MB block size

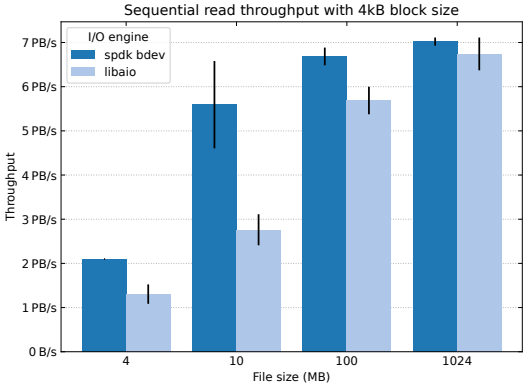


Figure 3.2: Throughput of sequential read operations with 4kB block size

Sequential write

For sequential write, the results are less conclusive with 4MB block size. For 4MB and 10MB file size, SPDK performs poorly, being 4 to 5 times slower than Linux kernel. Bigger file size For 4MB block size, SPDK mean throughput is 0.3 GB/s while Linux kernel is 1.52 GB/s. For 10MB block size, SPDK throughput is 0.66 GB/s while Linux kernel is 2.58 GB/s. 100MB and 1GB file size on the other side show a significant improvement of SPDK over Linux kernel, with respectively

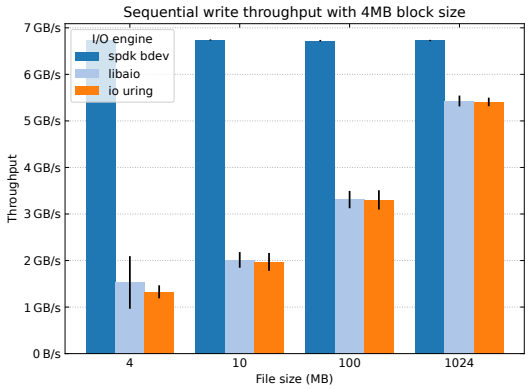


Figure 3.3: Throughput of sequential write operations with 4MB block size

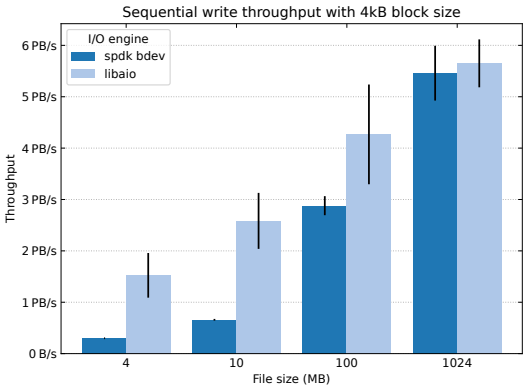


Figure 3.4: Throughput of sequential write operations with 4kB block size

3.1.2 Latency

Chapter 4

Appendix : performances script

This appendix references one of the NPF script used to evaluate performances in chapter 3, as well the Fish script to automate the evaluation process. One can easily find the scripts in the repository of the thesis, but they are also included here for completeness.

Sequential Read Script

Sequential Read Script

Listing 4.1: NPF script for sequential read throughput measurements with 4MB block size

```
%info

spdk-fio sequential read benchmark

%config

n_runs=10
var_names={BANDWIDTH:Throughput,IODEPTH:I/O Depth,ENGINE:I/O
↪  engine,SIZE:File size (MB)}
var_aggregate={IODEPTH:all}
var_unit={BANDWIDTH:B/s,SIZE:B}
var_divider={BANDWIDTH:1,SIZE:0.000001}
var_format={BANDWIDTH:eng.0,SIZE:eng.0.B}
graph_serie=ENGINE
graph_type=barplot

```

```
%variables
RW={read}
IODEPTH={1,16,32}
SIZE={4,10,100,1024}
ENGINE={spdk_bdev,libaio}

%script@server sudo=true

if [[ "${ENGINE}" == "spdk_bdev" ]];
then
    if [ -b "/dev/nvme0n1" ]; then
        PCI_ALLOWED="0000:02:00.0"
        ↪  /etinfo/users/2021/njeanmenne/spdk/scripts/setup.sh
        sleep 5
    fi
    LD_PRELOAD=/etinfo/users/2021/njeanmenne/spdk/build/fio/spdk_bde
    ↪  v fio --name=npf_spdk-fio_read
    ↪  \
              --ioengine=${ENGINE} \
              --spdk_json_conf=/etinfo/users/2021/njeanmenne/master-
              ↪  thesis/src/simple-config.json
              ↪  \
              --filename=Nvme0n1 \
              --direct=1 \
              --thread=1 \
              --rw=${RW} \
              --bs=4M \
              --iodepth=${IODEPTH} \
              --size=${SIZE}M \
              --output-format=json > fio.json
else

    if [ ! -b "/dev/nvme0n1" ]; then
        /etinfo/users/2021/njeanmenne/spdk/scripts/setup.sh reset
        sleep 5
    fi

    fio --name=npf_spdk-fio_read \
              --ioengine=${ENGINE} \
              --filename=/dev/nvme0n1 \
              --direct=1 \
              --rw=${RW} \
```

```
                --bs=4M \
                --iodepth=${IODEPTH} \
                --size=${SIZE}M \
                --output-format=json > fio.json
fi

if [[ "${RW}" == *"read"* ]];
then
    bw_bps=$(jq '.jobs[0].read.bw_bytes' fio.json)
else
    bw_bps=$(jq '.jobs[0].write.bw_bytes' fio.json)
fi

echo "RESULT-BANDWIDTH $bw_bps"
```

Automation script

Listing 4.2: Automation script for running NPF experiments

```
#!/usr/bin/env fish

argparse c/cache m/mode= s/suffix= -- $argv
or exit 1

set npf_file "$argv[1]"
set target_dir (dirname $npf_file)

set cache
if set -q _flag_cache
    set cache --cache
end

if set -q _flag_mode
    set mode (string split " " "$_flag_mode")
else
    set mode read write randread randwrite rw randrw
end

if set -q _flag_suffix
    set title_suffix $_flag_suffix
end
```

```
23
24  function sed_mode -a m file
25      sed -E -e "s/RW=[{][^}]*[}]/RW={$m}/g" \
26          -e "s/spdk_bdev fio \[.*\] benchmark/spdk_bdev fio [$m]
            ↪  benchmark/g" \
27          "$file" >"run.npf"
28  end
29
30  function run_npf_tests
31      if not test -f "$npf_file"
32          echo "Error: file '$npf_file' not found."
33          return 1
34      end
35
36      for rw in $mode
37          if test "$rw" = randread
38              set graph_title "Random read throughput"
39              sed_mode "$rw" "$npf_file"
40          else if test "$rw" = read
41              set graph_title "Sequential read throughput"
42              sed_mode "$rw" "$npf_file"
43          else if test "$rw" = randwrite
44              set graph_title "Random write throughput"
45              sed_mode "$rw" "$npf_file"
46          else if test "$rw" = write
47              set graph_title "Sequential write throughput"
48              sed_mode "$rw" "$npf_file"
49          else if test "$rw" = rw
50              set graph_title "Sequential mixed read/write"
51              sed_mode "$rw" "$npf_file"
52          else if test "$rw" = randrw
53              set graph_title "Random mixed read/write"
54              sed_mode "$rw" "$npf_file"
55          else
56              echo (set_color red) "Unrecognized paramater : $rw"
57              exit 1
58          end
59
60          if test -n "$title_suffix"
61              set graph_title "$graph_title $title_suffix"
62          end
63
```

```
        npf $cache --test "run.npf" --use-local $target_dir \
            --single-output "$target_dir/csv/$rw.csv" \
            --graph-title "$graph_title" \
            --graph-filename "$target_dir/graphs/$rw.pdf"

        rm "run.npf"
    end
end

run_npf_tests
```

Bibliography

[1] "NVMe over fabrics (oF) specification (historical reference only) - NVM express." (Jul. 12, 2021), [Online]. Available: https://nvmexpress.org/specification/nvme-of-specification/ (visited on 11/03/2025).

[2] "What is NVMe over fabrics? \textbar SNIA \textbar experts on data." (), [Online]. Available: https://www.snia.org/education/what-is-nvme-of (visited on 11/03/2025).

[3] Z. Guz, H. H. Li, A. Shayesteh, and V. Balakrishnan, "NVMe-over-fabrics performance characterization and the path to low-overhead flash disaggregation," in *Proceedings of the 10th ACM International Systems and Storage Conference*, ACM, May 22, 2017, pp. 1–9, ISBN: 978-1-4503-5035-8. DOI: 10.1145/3078468.3078483. [Online]. Available: https://dl.acm.org/doi/10.1145/3078468.3078483 (visited on 10/18/2025).

[4] R. Lin, Y. Cheng, M. D. Andrade, L. Wosinska, and J. Chen, "Disaggregated data centers: Challenges and trade-offs," *IEEE Communications Magazine*, vol. 58, no. 2, pp. 20–26, Feb. 2020, ISSN: 1558-1896. DOI: 10.1109/MCOM.001.1900612. [Online]. Available: https://ieeexplore.ieee.org/document/8999422 (visited on 11/02/2025).

[5] R. J. Recio, P. R. Culley, D. Garcia, B. Metzler, and J. Hilland, "A remote direct memory access protocol specification," Internet Engineering Task Force, Request for Comments RFC 5040, Oct. 2007. DOI: 10.17487/RFC5040. [Online]. Available: https://datatracker.ietf.org/doc/rfc5040 (visited on 11/03/2025).

[6] "RDMA consortium." (), [Online]. Available: https://www.rdmaconsortium.org/ (visited on 04/26/2026).

[7] "RDMA over converged ethernet," NVIDIA Docs. (), [Online]. Available: https://docs.nvidia.com/doca/sdk/RDMA-over-Converged-Ethernet/index.html (visited on 04/27/2026).

[8] "Storage performance development kit." (), [Online]. Available: https://spdk.io/ (visited on 11/02/2025).

[9] A. Skiadopoulos, Z. Xie, M. Zhao, *et al.*, "High-throughput and flexible host networking for accelerated computing," presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 405–423, ISBN: 978-1-939133-40-3. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/skiadopoulos (visited on 10/19/2025).

[10] X. Sun, M. Zhang, Y. Shan, K. Chen, J. Jiang, and Y. Wu, "Scalio: Scaling up DPU-based JBOF key-value store with NVMe- oF target offload," presented at the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), 2025, pp. 449–464, ISBN: 978-1-939133-47-2. [Online]. Available: https://www.usenix.org/conference/osdi25/presentation/sun (visited on 10/18/2025).

[11] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the TCP stack for i/o-offloaded content delivery," presented at the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), 2023, pp. 275–292, ISBN: 978-1-939133-33-5. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/kim-taehyun (visited on 10/19/2025).

[12] E. Jeong, S. Wood, M. Jamshed, *et al.*, "mTCP: A highly scalable user-level TCP stack for multicore systems," presented at the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14), 2014, pp. 489–502, ISBN: 978-1-931971-09-6. [Online]. Available: https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/jeong (visited on 11/24/2025).

[13] T. Barbette, *Tbarbette/npf*, Mar. 14, 2026. [Online]. Available: https://github.com/tbarbette/npf (visited on 04/25/2026).

[14] "Kernel/git/axboe/fio.git - fio - flexible IO tester." (), [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/axboe/fio.git/ (visited on 04/25/2026).

[15] "Overview - libaio - pagure.io." (), [Online]. Available: https://pagure.io/libaio (visited on 04/24/2026).

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain
Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | **www.uclouvain.be/epl**